



ATIVIDADE PRÁTICA: DESAFIOS DE ENGENHARIA

Instruções Gerais:

1. Abra o arquivo `main.py` contendo o código base do simulador de RPG.
 2. Analise a estrutura atual do código antes de iniciar as modificações.
 3. Os desafios possuem níveis progressivos de complexidade. Gerencie bem o seu tempo!
-

DESAFIO 1: Validação de Entrada (Gasto Obrigatório de Pontos)

- **Contexto:** No sistema atual, o jogador pode avançar na criação de personagem mesmo que decida não gastar todos os 20 pontos disponíveis.
 - **Sua Missão:** Modifique a função `criar_personagem` para que o jogo obrigue o usuário a gastar **exatamente os 20 pontos**. Se sobrarem pontos ou se o total for ultrapassado, o sistema deve exibir uma mensagem de erro e reiniciar a distribuição.
 - **Dica de Engenharia:** Verifique a condição do laço de repetição ou adicione uma validação condicional após o preenchimento de todos os atributos.
-

DESAFIO 2: Alteração de Estrutura (Novo Atributo "Defesa")

- **Contexto:** O sistema de esquiva atual considera apenas o bônus de *Destreza* para calcular a Classe de Armadura (CA). Queremos tornar o jogo mais estratégico adicionando uma camada extra de proteção.
 - **Sua Missão:**
 1. No dicionário de atributos da função `criar_personagem`, adicione uma nova chave chamada `"Defesa"`.
 2. Na inicialização do personagem (`__init__`), altere a fórmula da Classe de Armadura para:
`CA = 10 + bônus de Destreza + bônus de Defesa`
 - **Dica de Engenharia:** Lembre-se de ajustar a lógica de pontos do *Desafio 1*, pois agora os 20 pontos serão divididos entre 7 atributos, e não mais 6.
-

DESAFIO 3: Lógica Condicional Avançada (Mecânica de Fúria)

- **Contexto:** Para deixar as batalhas mais emocionantes, queremos criar um efeito de reviravolta quando um personagem estiver prestes a derrotado.
- **Sua Missão:** Vá até a função `simular_batalha` e altere a lógica de cálculo do Dano Físico (Opção 1). Caso o **HP atual** do atacante seja **menor que 10**, o dano

calculado pelo dado deve ser **multiplicado por 2** (Dano Dobrado). Exiba uma mensagem no terminal avisando que o personagem entrou em "Modo Fúria".

- **Dica de Engenharia:** Use uma estrutura condicional `if` antes de aplicar o dano final para verificar o status de vida do `atacante`.

DESAFIO 4 (AVANÇADO): Acumuladores e Relatório Final

- **Contexto:** Ao final do jogo, queremos saber quem foi o participante mais agressivo da partida, independentemente de quem venceu a luta.
- **Sua Missão:**
 1. Crie uma variável para rastrear o dano total de cada personagem. Ela deve começar em `0`.
 2. Toda vez que um ataque (físico ou mágico) for bem-sucedido e causar dano, acumule esse valor na variável do respectivo personagem.
 3. No final da função `simular_batalha` (junto com o anúncio do vencedor), exiba um "Relatório de Danos" mostrando o total acumulado por cada um e declarando quem causou mais dano.
- **Dica de Engenharia:** Use as `f-strings` com formatação de espaçamento (ex: `{:^35}` ou `{: <15}`) para que o relatório final mantenha a mesma estética visual de tabela que já existe no restante do projeto.

Boa sorte! Em caso de dúvidas ou erros de sintaxe, chame o monitor responsável.